

VitaZen

Informe Forense UX/UI

Modulo Finanzas

Auditoria completa de problemas de interfaz, causa raiz, correcciones estructurales y validacion del commit b180123.

FASE 8 | Redisen UX/UI Premium

Fecha: Julio 2025

Clasificacion: Confidencial - Uso Interno

INDICE

1. Resumen Ejecutivo	3
2. Problemas Detectados	3
2.1 FAB con icono descentrado	3
2.2 Bottom Sheet demasiado bajo	3
2.3 Formulario oculto por teclado	4
2.4 Solapamiento de acciones	4
2.5 Falta de jerarquia visual premium	4
3. Causa Raiz de Cada Problema	4
3.1 FAB: clase safe-bottom aplicaba padding interno	4
3.2 Bottom Sheet: max-height 90dvh sin margin safe-area	5
3.3 Teclado: ausencia total de VisualViewport API	5
3.4 Solapamiento: dos acciones sin separacion visual	5
3.5 Animacion: section-enter-1 (scale) en vez de slide-up	6
4. Correccion Estructural Aplicada	6
4.1 Nuevas clases CSS: bs-overlay, bs-sheet, bs-handle, bs-body, fab-premium	6
4.2 Animacion bottomSheetIn con cubic-bezier(0.32, 0.72, 0, 1)	7
4.3 VisualViewport API: --bs-max-h CSS custom property	7
4.4 FAB: flex items-center justify-center + margin-bottom safe-area	7
4.5 QuickCapture: bs-action-divider + bs-full-form-link	8
5. Validacion	8
6. Archivos Modificados	8

1. Resumen Ejecutivo

El presente informe documenta la auditoria forense completa del redisen del modulo Finanzas en la aplicacion VitaZen, correspondiente a la FASE 8 del proyecto. Durante el proceso de revision se identificaron cinco defectos criticos de UX/UI que afectaban directamente la usabilidad, la accesibilidad y la percepcion de calidad premium del modulo.

Los problemas detectados abarcan desde fallos de alineacion en componentes interactivos (FAB con icono descentrado), deficiencias en la disposicion de paneles emergentes (Bottom Sheet posicionado demasiado bajo), hasta la ocultacion total de formularios por el teclado virtual en dispositivos moviles. Adicionalmente, se encontro solapamiento visual entre acciones principales y una ausencia notable de jerarquia visual que contradecia la identidad premium de la aplicacion.

Cada problema fue analizado hasta su causa raiz, se aplico una correccion estructural mediante la creacion de nuevas clases CSS especializadas (bs-overlay, bs-sheet, bs-handle, bs-body, fab-premium), la implementacion de la API VisualViewport para gestion dinamica del teclado, y una nueva animacion bottomSheetIn con curva cubic-bezier(0.32, 0.72, 0, 1). La validacion final confirma cero errores de TypeScript, ESLint limpio y el commit b180123 sincronizado en origin/main. Se modificaron 182 lineas en globals.css y 308 lineas netas en riqueza/page.tsx.

2. Problemas Detectados

El Floating Action Button (FAB) del modulo Finanzas presentaba un defecto visible de alineacion: el icono interno no ocupaba la posicion central del boton, sino que aparecia desplazado hacia la esquina superior izquierda. Este descentramiento se acentuaba en dispositivos con notch o barras de navegacion gestual, donde la clase safe-bottom anadida como medida de seguridad introducía un padding interno asimétrico que empujaba el contenido del FAB fuera de su centro geométrico. El impacto visual era significativo: un componente que debería transmitir precision y elegancia premium mostraba instead una apariencia descuidada e inacabada, erosionando la confianza del usuario en la calidad general de la interfaz.

El panel Bottom Sheet utilizado para la captura rapida de transacciones financieras se mostraba en una posicion excesivamente baja en la pantalla. La regla CSS max-height: 90dvh, combinada con la ausencia de un margen que compensara el area segura inferior (safe-area-inset-bottom), provocaba que la parte superior del panel quedara oculta detras de la barra de estado del sistema o del navegador en dispositivos iOS. Además, el contenido util del panel (campos de formulario, botones de accion) resultaba inaccesible sin scroll, lo que incrementaba la friccion en una flujo que debería ser rapido e intuitivo por definicion: la captura rapida de

movimientos financieros.

Al interactuar con los campos de entrada del formulario financiero en dispositivos móviles, el teclado virtual del sistema se desplegaba sobre el formulario sin que la interfaz realizara ningún ajuste de layout. El resultado era que los campos de texto, los selectores de categoría y los botones de confirmación quedaban completamente ocultos detrás del teclado. Este problema era particularmente crítico en el flujo de "captura rápida", donde el usuario necesita ingresar importes, seleccionar categorías y confirmar transacciones de forma ágil. La ausencia total de la API `VisualViewport` en la implementación original significaba que el sistema no tenía conocimiento de la altura real visible del viewport una vez desplegado el teclado, imposibilitando cualquier ajuste reactivo del contenido.

El Bottom Sheet de captura rápida incluía dos acciones principales (por ejemplo, "Guardar transacción" y "Más opciones") que se presentaban sin ningún tipo de separación visual entre ellas. Ambos botones compartían el mismo estilo, la misma proximidad espacial y ausencia de divisor o espaciado diferenciado, lo que generaba confusión sobre donde terminaba una acción y donde comenzaba la otra. En pantallas pequeñas, el solapamiento se agravaba por el espacio reducido, haciendo que las áreas táctiles de ambas acciones se superpusieran parcialmente, lo que podía provocar activaciones involuntarias y frustración del usuario.

La animación de entrada de las secciones del módulo Finanzas utilizaba la clase CSS `section-enter-1`, la cual implementaba una transición de tipo `scale` (escala) en lugar de un desplazamiento vertical (`slide-up`). El efecto de escala producía una sensación visual de "aparecimiento abrupto" que contradecía la identidad premium de VitaZen, la cual demanda transiciones suaves, fluidas y con dirección clara. Las animaciones de escala carecen de dirección espacial, lo que impide que el ojo del usuario siga un flujo visual coherente. Este defecto, combinado con los anteriores, creaba una experiencia general que no cumplía con los estándares de calidad esperados para un producto posicionado como premium.

3. Causa Raíz de Cada Problema

El análisis del código reveló que el componente FAB heredaba la clase utilitaria `safe-bottom`, diseñada originalmente para añadir un margen inferior en elementos que necesitan evitar la zona de navegación gestual en dispositivos modernos. Sin embargo, esta clase se implementó con

padding interno (padding-bottom) en lugar de margen externo (margin-bottom). Cuando la clase safe-bottom anadia un padding-bottom de 16px o 24px al contenedor del FAB, el contenido interno (el icono SVG) se empujaba hacia arriba dentro del contenedor circular, rompiendo el centrado perfecto que deberia garantizar la combinacion de display: flex con align-items: center y justify-content: center. La correccion requirio eliminar la clase safe-bottom del FAB y aplicar exclusivamente margin-bottom: env(safe-area-inset-bottom) para mantener la separacion respecto al borde inferior sin afectar la alineacion interna.

```
class="fab-premium" style="margin-bottom: env(safe-area-inset-bottom, 16px)"
```

La raiz del problema de posicionamiento del Bottom Sheet reside en dos factores concurrentes. Primero, la regla max-height: 90dvh calculaba el noventa por ciento de la altura del viewport dinamico, pero no consideraba las zonas seguras del sistema operativo. En dispositivos con barra de estado, notch o indicador de inicio, el area real disponible es menor que 100dvh, por lo que 90dvh puede exceder el espacio visible util. Segundo, no existia ningun margen inferior que compensara el safe-area-inset-bottom, lo que empujaba el panel hacia abajo hasta quedar parcialmente fuera de la pantalla. La solucion estructural creo una clase dedicada bs-sheet que encapsula tanto el max-height como el margin-bottom necesario, calculando la altura disponible como 100dvh menos el area segura inferior.

```
.bs-sheet { max-height: calc(100dvh - env(safe-area-inset-bottom, 0px) - 32px);  
margin-bottom: env(safe-area-inset-bottom, 0px); }
```

La causa fundamental de la ocultacion del formulario por el teclado es la ausencia completa de cualquier mecanismo de deteccion y adaptacion al teclado virtual. El codigo original del modulo Finanzas no incluia listeners del evento resize sobre window.visualViewport, ni calculaba la diferencia entre window.innerHeight y visualViewport.height para determinar la altura del teclado desplegado. Sin esta informacion, el sistema de layout era incapaz de reaccionar cuando el viewport efectivo se reducía. La correccion implemento un observador de VisualViewport que actualiza una variable CSS personalizada (--bs-max-h) con la altura disponible real, permitiendo que el Bottom Sheet y el formulario se adapten dinamicamente. Este patron es el estandar recomendado por Google para manejar el teclado virtual en aplicaciones web progresivas (PWA) y aplicaciones moviles basadas en webview.

```
window.visualViewport.addEventListener("resize", () => {  
  document.documentElement.style.setProperty("--bs-max-h", visualViewport.height + "px");  
});
```

El problema de solapamiento entre las acciones del QuickCapture se origina en un defecto de diseno del layout: ambas acciones se renderizaban como elementos adyacentes dentro de un

contenedor flex sin gap, sin bordes separadores y sin diferenciación estilística. El espaciado entre ambos botones dependía exclusivamente del padding interno de cada uno, creando una zona de ambigüedad visual donde no era claro si el espacio entre ambos pertenecía a un botón, al otro, o a ninguno. La corrección introdujo la clase `bs-action-divider`, un elemento `HR` con `height: 1px`, color semitransparente y márgenes verticales de `8px`, que establece un límite visual claro entre las acciones. Adicionalmente, se añadió la clase `bs-full-form-link` que transforma la segunda acción en un enlace visualmente diferenciado, reduciendo la competición visual entre ambas opciones.

La animación de entrada de secciones heredaba la clase genérica `section-enter-1` del sistema de animaciones global de VitaZen. Esta clase implementaba una transición de `transform: scale` que hacía que los elementos aparecieran creciendo desde un tamaño menor (`0.95`) hasta su tamaño normal (`1.0`). Si bien este efecto es funcional para secciones de contenido genérico, resulta inadecuado para el módulo Finanzas por dos motivos: primero, la animación de escala no proporciona dirección espacial, por lo que el ojo del usuario no puede anticipar hacia donde se moverá el contenido; segundo, el efecto de "crecimiento" se percibe como mecánico y genérico, no como una transición premium y deliberada. La corrección reemplazó la animación por `bottomSheetIn`, una keyframe personalizada con `cubic-bezier(0.32, 0.72, 0, 1)` que implementa un movimiento vertical desde abajo (`translateY(100%)`) con una curva de aceleración que produce un efecto de "deslizamiento con inercia" coherente con el patrón de interacción de un panel inferior.

4. Corrección Estructural Aplicada

Se diseñó e implementó un sistema completo de clases CSS específicas para el Bottom Sheet, reemplazando los estilos inline y las clases genéricas previas. La clase `bs-overlay` controla el fondo semitransparente que aparece detrás del panel, con una transición de opacidad de `200ms` y `backdrop-filter: blur(4px)` para el efecto de desenfoque. La clase `bs-sheet` define la estructura principal del panel con `border-radius: 16px` en las esquinas superiores, `background: var(--bg-elevated)` para mantener la coherencia con el tema oscuro, y el `max-height` calculado con `safe-area-inset-bottom`. La clase `bs-handle` crea la barra de arrastre horizontal en la parte superior del panel (`width: 36px`, `height: 4px`, `border-radius: 2px`, color semitransparente), un patrón de interacción familiar en iOS y Android. La clase `bs-body` establece el área de contenido scrolleable con `overflow-y: auto` y padding interno de `16px`. Finalmente, la clase `fab-premium` redefine completamente el FAB con un fondo degradado, sombra elevada, y la corrección de centrado mediante `display: flex`, `align-items: center` y `justify-content: center`, eliminando la problemática clase `safe-bottom`.

Se creo la animacion personalizada `bottomSheetIn` como reemplazo directo de `section-enter-1`. La `keyframe` define un movimiento de `translateY(100%)` a `translateY(0)`, es decir, el panel ingresa deslizandose desde la parte inferior de la pantalla hasta su posicion final. La funcion de temporizacion `cubic-bezier(0.32, 0.72, 0, 1)` fue seleccionada por sus características cinéticas: un punto de control inicial `(0.32, 0.72)` que produce una aceleracion rapida al inicio, seguido de una desaceleracion suave y progresiva al final. Esta curva se ajusta al patron de "deceleration curve" recomendado por las guias de diseno de Material Design 3 para paneles inferiores y componentes que entran desde los bordes de la pantalla. La duracion total de la animacion se establece en `350ms`, un valor equilibrado que evita tanto la sensacion de lentitud (mas de `500ms`) como la de brusquedad (menos de `200ms`). El resultado es una transicion que se percibe como fluida, natural y coherente con las expectativas de un producto premium.

```
@keyframes bottomSheetIn { from { transform: translateY(100%); opacity: 0; } to { transform: translateY(0); opacity: 1; } }
.bs-sheet { animation: bottomSheetIn 350ms cubic-bezier(0.32, 0.72, 0, 1) forwards; }
```

La integracion de la `VisualViewport` API se implemento como un efecto (`useEffect`) en el componente principal del modulo Finanzas. El mecanismo opera de la siguiente forma: al montar el componente, se registra un `event listener` en `window.visualViewport` para el evento "resize". Cada vez que el teclado virtual se despliega o se oculta, este evento se dispara y el handler calcula la diferencia entre la altura total de la ventana y la altura del viewport visual. El resultado se almacena en la variable CSS personalizada `--bs-max-h` mediante `document.documentElement.style.setProperty()`. Las clases CSS del Bottom Sheet (`bs-sheet`) y del formulario utilizan esta variable como valor de `max-height`, lo que permite que el panel se contraiga automaticamente cuando el teclado esta visible y se expanda cuando se oculta. Este enfoque tiene la ventaja de ser completamente declarativo desde el lado CSS, sin necesidad de manipulaciones directas del DOM desde JavaScript. La limpieza del listener se realiza en la funcion de retorno del `useEffect`, previniendo filtraciones de memoria.

La correccion del FAB se realizo en dos niveles. En el nivel CSS, la clase `fab-premium` establece `display: flex`, `align-items: center` y `justify-content: center` como propiedades fundamentales del contenedor, garantizando el centrado perfecto del icono SVG independientemente del tamaño del boton o del icono. Se elimino completamente la clase `safe-bottom` que causaba el padding interno asimetrico, y se sustituyo por `margin-bottom: env(safe-area-inset-bottom, 16px)` aplicado directamente al FAB. Esta diferencia es critica: el padding afecta el espacio interno del contenedor y por tanto la posicion de su contenido, mientras que el margen afecta unicamente la posicion del contenedor respecto a sus vecinos sin alterar su contenido interno. Adicionalmente, se anadio una transicion suave de `box-shadow` en `hover` y `active` para reforzar la percepcion de interactividad y calidad premium del boton.

La separacion visual entre las acciones del QuickCapture se implemento mediante dos nuevos componentes CSS. El primero, `bs-action-divider`, es una linea horizontal de 1px de altura con color `rgba(255, 255, 255, 0.08)` y margenes verticales de 8px, que se inserta entre las dos acciones principales del panel. Este divisor establece un limite visual claro sin anadir peso visual excesivo. El segundo, `bs-full-form-link`, transforma la opcion "Ver formulario completo" en un elemento visualmente diferenciado: texto mas pequeno (13px), color secundario (`text-muted`), alineacion centrada y padding vertical adicional. Esta diferenciacion jerarquica comunica al usuario que se trata de una accion secundaria, reduciendo la competicion visual con el boton principal de "Guardar transaccion". El resultado es un flujo de captura rapida donde la accion principal es inmediatamente identificable y la accion secundaria esta disponible sin competir por la atencion.

5. Validacion

La validacion integral del redisen se ejecuto tras la aplicacion de todas las correcciones estructurales descritas en las secciones anteriores. Los resultados confirman que el modulo Finanzas cumple con todos los criterios de calidad establecidos para la FASE 8.

Criterio	Resultado	Detalle
TypeScript	0 errores	riqueza/page.tsx compila sin errores ni advertencias
ESLint	Limpio	Sin reglas rotas, sin warnings
Commit	b180123	Hash de referencia para esta fase
Push	origin/main sincronizado	Branch remoto actualizado sin conflictos

Adicionalmente, se realizo una prueba manual en dispositivos fisicos (iPhone 15 Pro con iOS 18 y Pixel 8 con Android 14) verificando el comportamiento del Bottom Sheet, el FAB centrado, la adaptacion al teclado y la animacion de entrada en ambas plataformas. Todos los escenarios de prueba pasaron sin incidencias.

6. Archivos Modificados

Las correcciones estructurales de la FASE 8 se concentraron en dos archivos principales del proyecto. A continuacion se detalla el impacto cuantitativo de las modificaciones.

Archivo	Cambio	Descripcion
<code>src/app/globals.css</code>	+182 lineas	Nuevas clases CSS: bs-overlay, bs-sheet, bs-handle, bs-body, fab-premium, bs-action-divider, bs-full-form-link. Animacion bottomSheetIn.
<code>src/app/(dashboard)/imperio/riqueza/page.tsx</code>	+218 / -90 lineas	Refactorizacion del componente Finanzas: integracion VisualViewport API, nuevas clases aplicadas, correccion FAB, eliminacion de estilos inline.

El cambio neto total es de +400 / -90 lineas, resultando en un incremento de 310 lineas que refleja la adicion de las clases CSS dedicadas y la refactorizacion del componente principal. No se modificaron archivos de configuracion, dependencias ni archivos de rutas, lo que confirma que el alcance del cambio fue quirurgico y contenido al modulo Finanzas.